Randomized Data Structure : Skip List

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

November 26, 2024

Overview

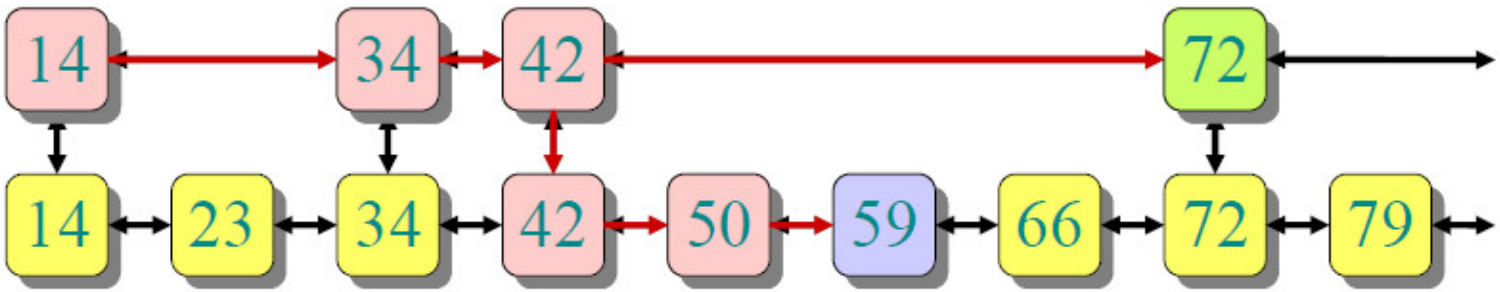
1. Skip List
2. Books etc.

1

[1] Slide Courtesy Erik D. Demaine and Charles E. Leiserson, 2001

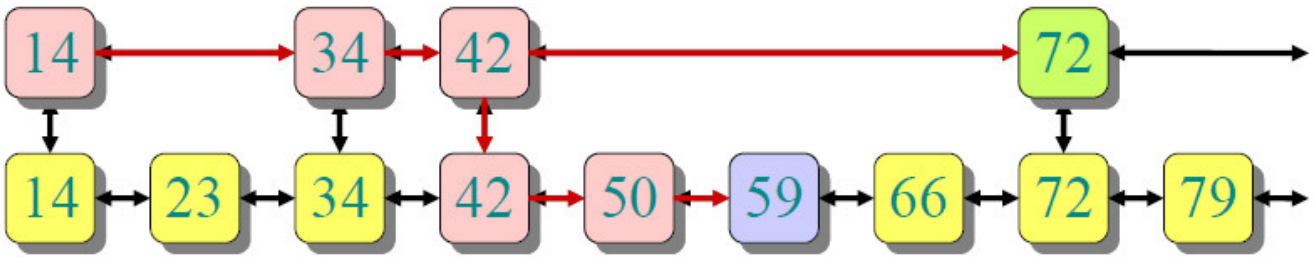
Overview: Skiplist

- Skip list is an efficient dynamic Search Structure, invented by William Pugh in 1989.
- AVL tree gives $O(\log n)$ worst case search/ insert/ delete time, rotation operations need to be done carefully
- Thus, **Skip List**, easier to implement but has $O(\log n)$ expected running time for search, insert, delete with **high probability**
- **Skip list** is a randomized data structure that has a set of connected sorted linked lists
- Example: Look at galloping and local regular trains; to reach faster to a station, we take a galloping train, that does not stop at every station on the way.



Skip List : Search

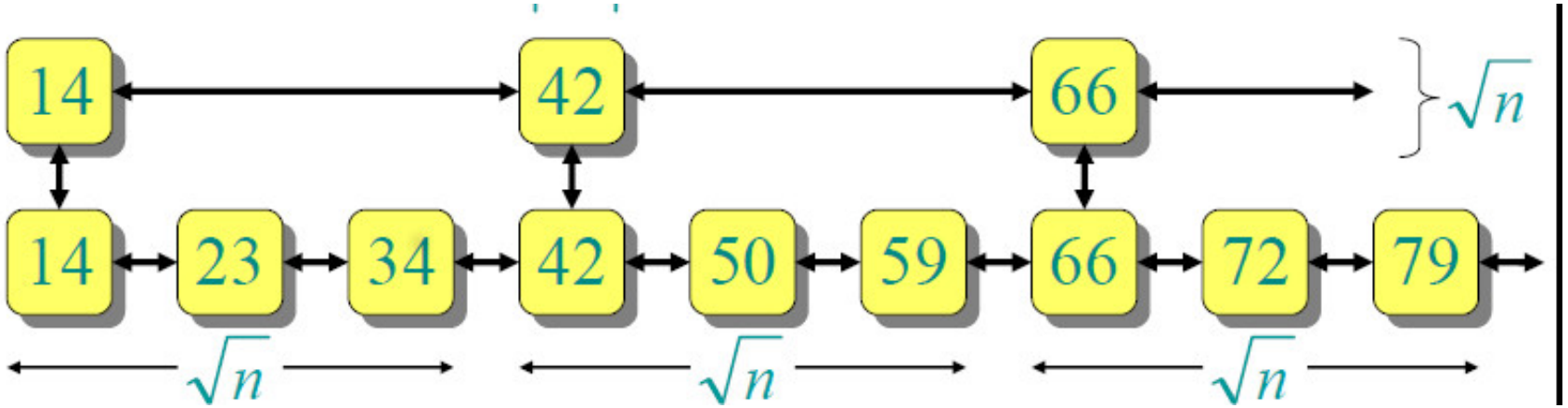
- **Express** line connects a few of the stations, **local** line connects all stations, **Links** between lines at common stations
- Search(x):
  - Walk right in top linked list ($L1$) until going right would go too far
  - Walk down to bottom linked list ($L2$)
  - Walk right in $L2$ until element found (or not)
- Search(59): Start from left go till search key 59 is greater than the keys at node;$59 < 72$; Take the link from 42 down and traverse right until key is found or list $L2$ is exhausted (search key not found
- Note, the lists are doubly linked, bottom list contains all keys, some keys of bottom list appear in top list, but not all and corresponding nodes of two lists are doubly linked.



Analysis of Search Cost

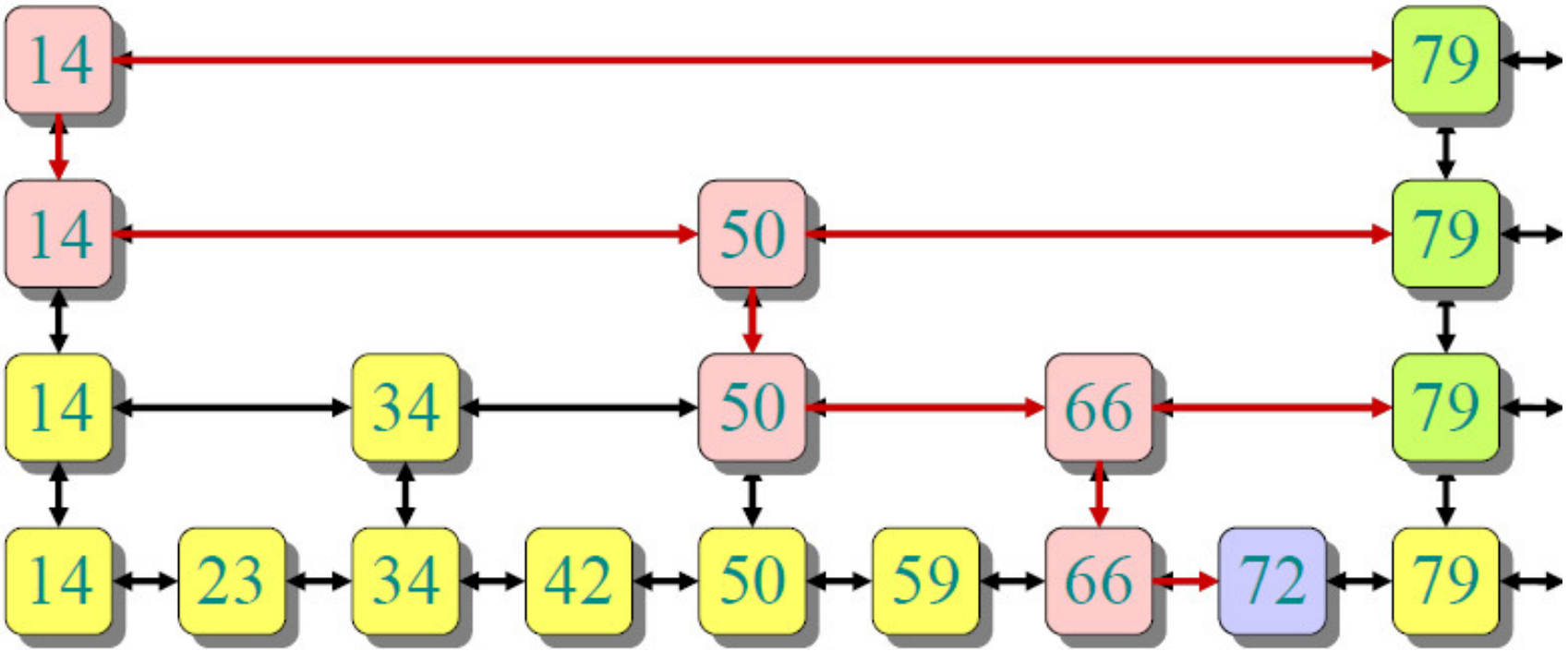
Question: Which keys should be in the top lists?

- Search Cost : $|L1| + \frac{|L2|}{|L1|} = |L1| + \frac{n}{|L1|}$
- Cost is minimized, when $|L1| = \frac{n}{|L1|} \implies |L1|^2 = n \implies |L1| = \sqrt{n}$
- Thus, Search Cost : $|L1| + \frac{n}{|L1|} = 2\sqrt{n}$



- 2 sorted list search cost: $2\sqrt[2]{n}$
- 3 sorted list search cost: $3\sqrt[3]{n}$
- k sorted list search cost: $k\sqrt[k]{n}$
- $\log n$ sorted list search cost: $\log n \sqrt[\log n]{n} = \log n.n^{\frac{1}{\log n}} = \log n.n^{\frac{\log 2}{\log n}} = \log n.n^{\log_n 2} = \log n.2 = \Theta(\log n)$

Ideal Skip List



- log n sorted linked list the **ideal skip list** which is just like Binary Tree
- Search(72): Start from top list, finds the key only at the bottom list;
  $(L1:)14 \rightarrow 79 \leftarrow 14 \downarrow$, $(L2:)14 \rightarrow 50 \rightarrow 79 \leftarrow 50 \downarrow$,
  $(L3:)50 \rightarrow 66 \rightarrow 79 \leftarrow 66 \downarrow$, $(L5:)66 \rightarrow 72$

Insert (x) into skip list

- Search(x) to see where x fits in bottom list
- Always insert into bottom list; bottom list contains all elements
- Insert into some of the lists above

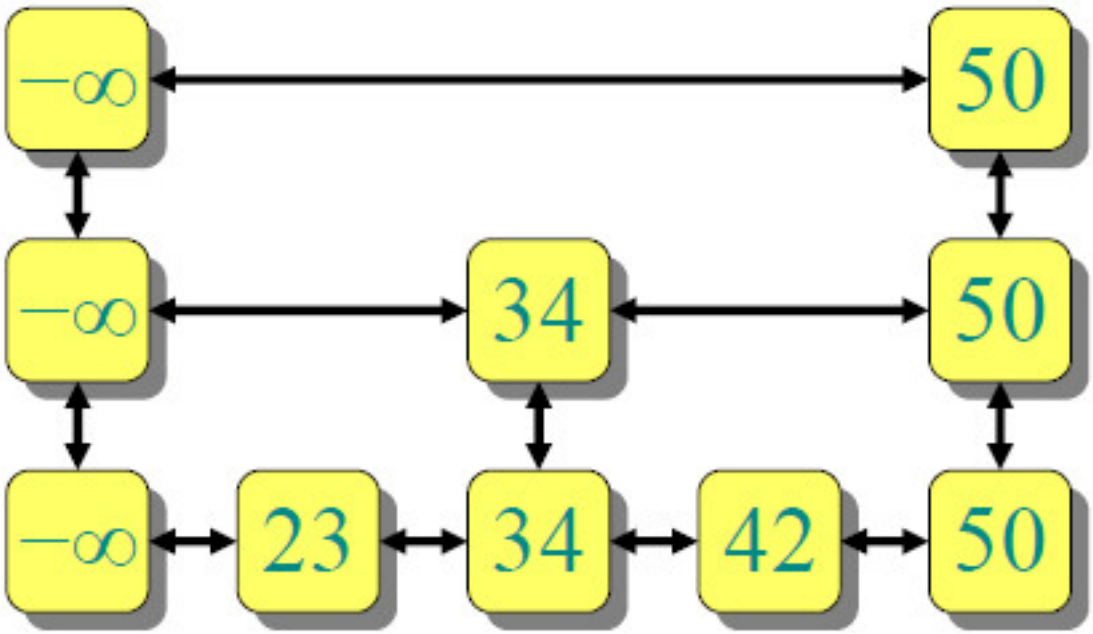
Question: To which other lists to add x?
Answer: Flip a (fair) coin; if HEADS, promote x to next level up and flip again until TAIL comes.

- Note that probability of a head, ie, x coming to next upper level is $1/2$
- On average $1/2$ of elements promoted to level 1, then $1/4$ to level 2, $1/8$ to level 3 and so on.

Skip List: Summary

- A skip list is the result of insertions (and deletions) from an initially empty structure (containing just ∞)
- INSERT(x) uses **random coin flips** to decide promotion level
- DELETE(x) removes x from all lists containing it

Add ∞ to every list for easy implementation



How good are skip lists? very good almost always

With high probability theorem

Theorem: With high probability, every search in an n element skip list costs $O(\log n)$

- **Informaly:** Event E occurs **with high probability (w.h.p.)** if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which E occurs with probability at least $1 - O(1/n^{\alpha})$; constant in $O(\log n)$ depends on α
- **Formally:** Parameterized event E_{α} occurs with high probability if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which E_{α} occurs with probability $\geq 1 - c_{\alpha}/n^{\alpha}$
- Can make **error probability** $O(1/n^{\alpha})$ very small by setting α large, e.g., 100

Analysis (1)

Lemma: With high probability, n element skip list has $O(\log n)$ levels

- To prove, analyze search **backwards**- leaf to root
  - Search starts [ends] at leaf (node in bottom level)
  - At each node visited: **If** node **wasnt promoted higher** (got TAILS here), then we go [came from] left
  - **Else** If node was **promoted higher** (got HEADS here), then we go [came from] up
  - Search stops [starts] at the root (or ∞)
- Search makes **up** and **left** moves until it reaches the root (or $-\infty$)
- Compute number of **up** moves for number of levels
- Note that, a key is moved to new level as coin flip gives HEAD and promotes till a TAIL is found

Important Concepts to be used:

- **Booles' Inequality / Union Bound:**For any random events $E_1, E_2, \cdots, E_k$,
  $Pr[E_1 \cup E_2 \cup \cdots \cup U_k] \leq Pr[E_1] + Pr[E_2] + \cdots + Pr[E_k]$
- **Waiting for success bounds**

Analysis (2)

Proof:

- Let X_i: Random variable denoting number of trials (for i^{th} element) needed to perform to get a TAIL (ie, no further promotion of key)
- Let $X = \max_i X_i$ is independednt and identically distributed geometric random variable with parameter $p = 1/2$ (since coin toss)
- $Pr[X_i = k] = (1-p)^{k-1}p$
- $Pr[X_i \leq t]$

Analysis (3)

$$Pr[X_i \leq t]$$

$$= Pr[X_i = 1] + Pr[X_i = 2] + \cdots Pr[X_i = t]$$
$$= p + (1-p)p + \cdots + (1-p)^{t-1}p$$
$$= p(1 + 1 - p + (1-p)^{t-1})$$
$$= p.(1 + \frac{(1-p).(1-(1-p)^{t-1}}{1-(1-p)})$$
$$= p.(1 + \frac{(1-p)-(1-p)^{t}}{p})$$
$$= p + (1-p) - (1-p)^{t}$$
$$= 1 - (1-p)^{t}$$

- $Pr[X_i > t] = 1 - (1 - (1-p)^t) = (1-p)^t$
- Find $Pr[X > c \log n] \leq 1/n^{\alpha}$; Show that it is very low
- Thus, $Pr[X \leq c \log n] \geq (1 - 1/n^{\alpha})$
- $Pr[X > t] = Pr\left[\max_i X_i > t\right]$

Analysis (4)

$$Pr[X > t] = Pr\left[\max_i X_i > t\right]$$

$$= Pr[\text{at least one } X_i > t]$$

$$= Pr[(X_1 > t) \cup (X_2 > t) \cup \cdots (X_n > t)] \text{ Using Boole's inequality}$$

$$\leq \sum_{i=1}^{n} Pr[X_i > t]$$

$$\leq n(1-p)^t = n(1-1/2)^{c \log n} = \frac{n}{2^{c \log n}} = \frac{n}{n^c} = \frac{1}{n^{c-1}} = \frac{1}{n^\alpha}$$

- For sufficiently large c, thus, α, $\frac{1}{n^\alpha}$ is very small,ie, $Pr[X > c \log n] \leq \frac{1}{n^\alpha}$
- Thus, the Theorem is proved, ie, Number of levels, thus search, insert, delete takes $O(\log n)$ time with high probability,i.e, $1 - (1/n^\alpha)$
- The error probability is only $1/n^\alpha$.

References

 J. Kleinberg and E. Tardos, Algorithm Design,Pearson, Chapter 13.6,13.7

The End